

JavaScript Closures



Local Variables

A local variable is a "private" variable defined inside a function.

A function can access all variables in the local scope.

a is a local variable defined inside the function:

```
function myFunction() {  
  let a = 4;  
  return a * a;  
}
```


Global Variables

A global variable is a "public" variable defined outside a function.

A function can access all variables in the global scope:

a is global variable defined outside the function:



```
let a = 4;  
function myFunction() {  
    return a * a;  
}
```




IMPORTANT TO KNOW

Global variables can be used (or changed) by all scripts in the page.

A local variable can only be used inside the function where it is defined. It is private and hidden from other functions and other scripting code.

Global and local variables with the same name are different variables. Modifying one, does not modify the other.

Undeclared variables (created without a keyword var, let, const), are always global, even if they are created inside a function.



JavaScript Nested Functions

All functions have access to the global scope.

All functions have access to the scope "above" them.

Nested functions have access to the scope "above" them.

```
function add() {  
  let counter = 0;  
  function plus() {counter += 1;}  
  plus();  
  return counter;  
}
```

The inner function plus() has access to the counter variable in the parent function:



JavaScript Closures Example

```
function myCounter() {  
  let counter = 0;  
  return function() {  
    counter++;  
    return counter;  
  };  
}  
  
const add = myCounter();  
add();  
add();  
add();  
  
// the counter is now 3
```




Example Explained

The variable `add` is assigned to the return value of a function.

The function only runs once. It sets the counter to zero (0), and returns a function expression.

This way `add` becomes a function. The "wonderful" part is that it can access the counter in its parent scope.

This is called a closure. It makes it possible for a function to have "private" variables.

The counter is protected by the scope of the `myCounter` function, and can only be changed using the `add` function.

Example

```
function outer() {  
  let count = 0;  
  return function inner() {  
    count++;  
    console.log(count);  
  };  
}
```

 **outer**
defines a
variable count
and returns
inner.

```
const counter = outer();  
counter(); // 1  
counter(); // 2
```

This is a closure:
A function
remembering
variables from its
outer scope.

 **Even though outer() is done executing,**
inner() still accesses count

Why Use Closures?

- ✓ **Data privacy (like private variables)**
- ✓ **Preserve state across calls**
- ✓ **Useful in callbacks, async code, and more!**

Closures are everywhere in JS once you know what to look for.

Recap

 **A closure = Function + Lexical Environment**

 **It's one of the core concepts behind JS's flexibility.**

 **How have you used closures in your code?**

FOUND THIS HELPFUL?

FOLLOW FOR MORE.

